



SFC - Projekt

LSTM nebo GRU - porovnání s vícevrstevným
perceptronem, 1D konvoluční sítí nebo transformerem
při zpracování sekvenčních dat

1 Introduction

The goal of this project is to implement and compare several neural network architectures used in machine learning on sequential data. Specifically, we focus on four models: Multilayer Perceptron (MLP), 1D Convolutional neural network, Gated recurrent unit (GRU) and Transformers. We tested all of these models on two tasks, regression and classification. Each task was performed on a sequential dataset.

The following sections describe each architecture in detail, introduce the selected datasets, present the experimental setup and results, and conclude with a short user guide explaining how to run the experiments.

2 Datasets

This section outlines the datasets used in our experiments and specifies the tasks to which they were applied.

For the first task, sentiment classification, we evaluated all models on sequential textual data. The objective of this task is to correctly predict the sentiment label of a given text sample. So, for example, it should correctly predict whether the given product review is positive or negative. Our initial dataset was the IMDB movie reviews dataset¹, which contains about 50,000 positive or negative reviews.

After the initial experiments, we decided to further compare the models by picking a more complex dataset. We selected the Stanford Sentiment Treebank (SST5)² dataset used as a benchmark for evaluation of the state-of-the-art Transformer architectures. Unlike IMDB, SST5 uses five classes: very negative, negative, neutral, positive, and very positive. With roughly 10,000 samples, the dataset is significantly smaller and therefore more difficult for the models.

We further extended our experiments by performing experiments on different tasks, that is, regression on time series data. The first dataset we selected is the Daily Minimum Temperatures³. This dataset contains several years of daily temperature measurements.

As in the classification task, our aim was to introduce a more challenging dataset⁴. For this, we chose an electric power consumption forecasting dataset. Predicting energy usage should be a more complex task, making it a more demanding regression task for our model. In the following section, we provide a detailed description of each model architecture used in our experiments.

3 Models

In this section, we describe the implementation details of each model used in our experiments and briefly explain the underlying principles of how they work.

MLP

The first model we implemented was MLP. We selected it as our baseline architecture because it represents the simplest approach to sequence related tasks. It has no mechanisms for processing order or temporal dependencies. Therefore, our initial assumption was that it would perform the worst among all tested models.

To apply an MLP to textual data, we used a Bag of Words (BoW) [1] representation. This approach converts each input sentence into a fixed length vector where each element corresponds to the count of a particular word in the vocabulary. Although this representation ignores word order and contextual meaning, it allows the MLP to capture basic associations between word frequencies and output labels.

In our setup for sentiment classification, each input text was converted into its BoW representation and passed through the fully connected layers of the MLP. For the regression task, we adapted the same

¹<https://huggingface.co/datasets/stanfordnlp/imdb>

²<https://huggingface.co/datasets/SetFit/sst5>

³<https://www.kaggle.com/datasets/suprematism/daily-minimum-temperatures>

⁴<https://www.kaggle.com/datasets/uciml/electric-power-consumption-data-set>

architecture to operate on numerical time series data. Instead of BoW vectors, the input consisted of fixed length sliding windows extracted from the temperature sequence.

1D convolution

Regarding the second model, we implemented 1D CNN, based on Kim architecture [2] for sentence classification. Convolutional models operate by applying filters to local regions of the input sequence in time series data. In the context of text classification, the 1D CNN treats the sequence of token embeddings as a temporal signal. Convolutional filters slide over the sequence and learn to respond to informative patterns.

GRU

For the recurrent architecture, we chose to implement a GRU rather than an LSTM. GRUs are conceptually simpler, contain fewer parameters and generally train faster while still achieving performance comparable to LSTM [3].

GRUs address the limitations of vanilla recurrent networks by introducing gating mechanisms that regulate the flow of information through the hidden state. Specifically, the update and reset gates control how much of the past context is retained or discarded at each time step. These gating operations allow the GRU to capture long range dependencies more effectively than traditional RNNs, making it well suited for tasks involving sequential data such as text classification and time series prediction.

Transformer

For the final model, we implemented a Transformer based architecture, which has become one of the most widely used approaches for sequence modeling in recent years. Transformers rely on self attention mechanisms, which allow them to capture relationships between all elements in a sequence simultaneously, rather than processing inputs strictly in order as in recurrent networks [4].

In our implementation, each token in the input sequence is first converted into an embedding vector and combined with positional encoding to retain information about the order of tokens. The embeddings are then processed by a stack of Transformer encoder layers. Each layer applies multi head self attention to model interactions between tokens and a feed forward network to capture higher level representations.

4 Experiments

In this section, we present the experiments performed on the chosen datasets, along with the corresponding results. We first discuss the sentiment classification task and then present results for the regression task.

Sentiment classification

The initial experiments were performed on the IMDB movie reviews dataset. The primary goal was to evaluate how each model performs in predicting the sentiment of a given review, classified as positive or negative.

We were tracking training loss and validation loss over epochs. For each model, the checkpoint corresponding to the highest validation accuracy was selected as the best model.

After training, each model was evaluated on the test set using accuracy, precision, recall and F1-score, the results can be seen in the next Table 1:

Model	Test Accuracy	Precision	Recall	F1-score
MLP	0.8808	0.88	0.88	0.88
Kim CNN	0.8658	0.87	0.87	0.87
GRU	0.8828	0.88	0.88	0.88
Transformer	0.8484	0.85	0.85	0.85

Table 1: Test metrics for sentiment classification on the IMDB dataset.

From the results, we observe that all models achieve relatively high performance, with the GRU obtaining the highest test accuracy. The Transformer yields the lowest scores among the models, likely due to its higher capacity and the limited size of the IMDB dataset, which may not provide enough data for effective training. The strong performance of the MLP indicates that simple Bow features already capture substantial information for this binary sentiment classification task. The GRU’s slight advantage over the CNN suggests that modeling longer range dependencies across the entire review provides a benefit, whereas CNN captures more local patterns.

The second set of experiments was performed on the SST5 dataset, a more challenging sentiment classification task. Unlike the IMDB dataset, SST5 contains five sentiment categories: very negative, negative, neutral, positive, and very positive. In addition, the dataset is smaller, which increases the difficulty for all models.

As before, the evaluation metrics on the test set include accuracy, precision, recall, and F1-score. The results are summarized in Table 2:

Model	Test Accuracy	Precision	Recall	F1-score
MLP	0.4131	0.43	0.38	0.38
Kim CNN	0.3615	0.35	0.32	0.32
GRU	0.4000	0.43	0.37	0.37
Transformer	0.3285	0.34	0.28	0.26

Table 2: Test metrics for sentiment classification on the SST5 dataset.

The results for the SST5 dataset were considerably lower compared to those obtained on the IMDB dataset. The MLP and GRU performed best, while CNN and Transformer struggled significantly. This performance drop can be attributed primarily to the small size of the SST5 dataset and the increased difficulty of the five class classification task.

Given that the models relying on embeddings (CNN, GRU, Transformer) showed clear difficulties when trained from scratch, we conducted an additional set of experiments using pretrained GloVe embeddings, which were then fine tuned on the SST5 task. Since the MLP does not operate on embeddings, its performance remains unchanged and is included only for reference. The results of these experiments are shown in Table 3.

Model	Test Accuracy	Precision	Recall	F1-score
MLP	0.4131	0.43	0.38	0.38
Kim CNN	0.4217	0.38	0.38	0.38
GRU	0.4335	0.44	0.39	0.39
Transformer	0.4321	0.44	0.41	0.41

Table 3: Test metrics for sentiment classification on the SST5 dataset, with pretrained embedding weights.

Using pretrained GloVe embeddings led to improvements across the embedding based models on the SST5 dataset. The GRU benefited the most, achieving the highest accuracy. The CNN also improved and slightly outperformed the MLP, while the Transformer achieved the similar accuracy almost as GRU.

Overall, the results indicate that pretrained embeddings provide a meaningful boost, but the task remains challenging for all architectures.

Regression on time series data

Regression experiments were conducted on the Daily Minimum Temperatures dataset, with the goal of predicting future temperature values from historical observations. As in previous tasks, the model selected for the final evaluation was the one achieving the best validation performance during training.

After training, each model was assessed on the test set using Mean Absolute Error (MAE) and the coefficient of determination (R^2), providing complementary information on the absolute prediction error and the overall fit. In addition to numerical metrics, plots comparing the true and predicted temperature trajectories are included to visually evaluate how well each model captures temporal patterns and seasonal trends. The following Table 4 contains the results of the metrics for the regression task:

Model	MAE	R squared
MLP	0.0675	0.6270
Kim CNN	0.0769	0.5079
GRU	0.0661	0.6435
Transformer	0.0668	0.6337

Table 4: Test metrics for regression on the Daily Minimum Temperatures dataset.

Based on the results, the GRU achieved the best performance, with the lowest MAE and the highest R^2 , confirming its strength in modeling temporal dependencies. The Transformer followed closely, showing that attention mechanisms can also capture relevant seasonal and temporal patterns in the data. The MLP performed surprisingly well, coming close to the Transformer despite its simpler architecture, suggesting that the smooth structure of the data set does not always require complex sequence modeling. In contrast, CNN obtained the weakest results, likely because convolutional filters struggle to capture longer term temporal relationships. In general, models designed for sequence processing, particularly recurrent and attention based architectures, proved to be more effective for temperature forecasting.

Next, we evaluated the models on a second regression task using the Electric Power Consumption dataset, which should be more complex than the temperature series. The following Table 5 contains the results of the metrics for the regression task

Model	MAE	R squared
MLP	0.0520	0.5675
Kim CNN	0.0814	0.1525
GRU	0.0507	0.5690
Transformer	0.0494	0.5698

Table 5: Test metrics for regression on the electric power consumption dataset.

Across models, performance decreased compared to the temperature dataset, confirming the increased difficulty of this task. The Transformer achieved the lowest MAE and the highest R^2 , indicating that its attention mechanism is particularly effective for more irregular data. The GRU performed nearly as well, with slightly higher MAE and marginally lower R^2 , suggesting that it also handles sequential dependencies effectively. The MLP remained competitive, showing that even a simple architecture can extract useful patterns when the input structure is not too complex. In contrast, the Kim CNN performed significantly worse, highlighting its difficulty in handling long-range dependencies and the noise inherent in this dataset. Overall, the results suggest that attention-based models, particularly the Transformer, are highly effective for complex time series regression, excelling in both pointwise predictions and capturing the global structure of the data.

For both experiments of the regression task, we generated visual graphs that compare the true values with the predicted values for each model. Due to their large size, these plots have been included in the Appendices A and B.

Based on the results, we can see that our assumptions about the MLP were not entirely correct. In sentiment classification on the IMDB dataset, it was one of the best performing models, which makes sense given that review sentiment often correlates strongly with specific keywords. However, its weaknesses became apparent on the more challenging SST5 dataset, where models that better capture contextual or sequential information scaled more effectively with increased complexity. In the time series experiments, the MLP still produced solid results, but it consistently ranked in the lower half of the models.

The 1D CNN performed well on IMDB but showed the weakest results on the remaining tasks. The GRU achieved some of the strongest results in sentiment classification and regression, highlighting its advantage in modeling sequential dependencies. The Transformer showed mixed behavior. It performed better on regression tasks than on sentiment classification, but its sentiment performance improved with pretrained GloVe embeddings. Since our Transformer was a small custom implementation, it is expected that larger pretrained architectures like BERT or GPT like models would outperform all models in this project.

5 User Manual

This project uses Python 3.10 and is managed with Conda. For installation instructions and all required dependencies, please refer to `readme.md`. The easiest way to configure the environment is via the provided `environment.yml` file.

The `src` folder contains demonstration scripts for both tasks. To run these demonstrations, you need trained models. You have two options:

1. Run the training scripts, which train and evaluate the models.
2. Use pre-trained models available via the provided Google Drive links.

Note: Training can take significant time, especially on the CPU. The longest training runs are typically for the IMDB experiments, while other tasks usually complete within 20–30 minutes depending on your hardware.

To run a demo, for example, sentiment analysis on the IMDB dataset with the MLP model, execute the following:

```
python demo_sentiment.py --dataset imdb --model mlp
```

Similarly, regression demos can be run with `demo_regression.py`.

6 Conclusion

The goal of this work was to compare several neural network architectures commonly used for sequential data processing. We evaluated an MLP, a 1D convolutional network, a GRU, and a Transformer model. To make the comparison more robust, we decided to compare the models on textual and numerical data. We used text data for sentiment classification and numerical data for next step time series prediction. The experiments showed that the MLP performed surprisingly well, exceeding the initial expectations. The 1D convolutional network was consistently among the weakest models in our setup. The GRU achieved strong and stable performance across all tasks, while the Transformer produced mixed results.

For future work, it would be beneficial to evaluate these architectures on larger and more challenging datasets, where models such as the GRU and Transformer are likely to gain a clearer advantage and where the limitations of the MLP may become more visible. Further hyperparameter tuning and the use of more advanced architectures, especially different Transformer models like BERT or GPT could also provide deeper insights into this work.

References

- [1] Eda Kavlakoglu Jacob Murel. *What is bag of words?* 2025. URL: <https://www.ibm.com/think/topics/bag-of-words>.
- [2] Yoon Kim. *Convolutional Neural Networks for Sentence Classification*. 2014. arXiv: 1408.5882 [cs.CL]. URL: <https://arxiv.org/abs/1408.5882>.
- [3] Junyoung Chung et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. arXiv: 1412.3555 [cs.NE]. URL: <https://arxiv.org/abs/1412.3555>.
- [4] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.

A Plots of true value vs predicted for daily minimum dataset

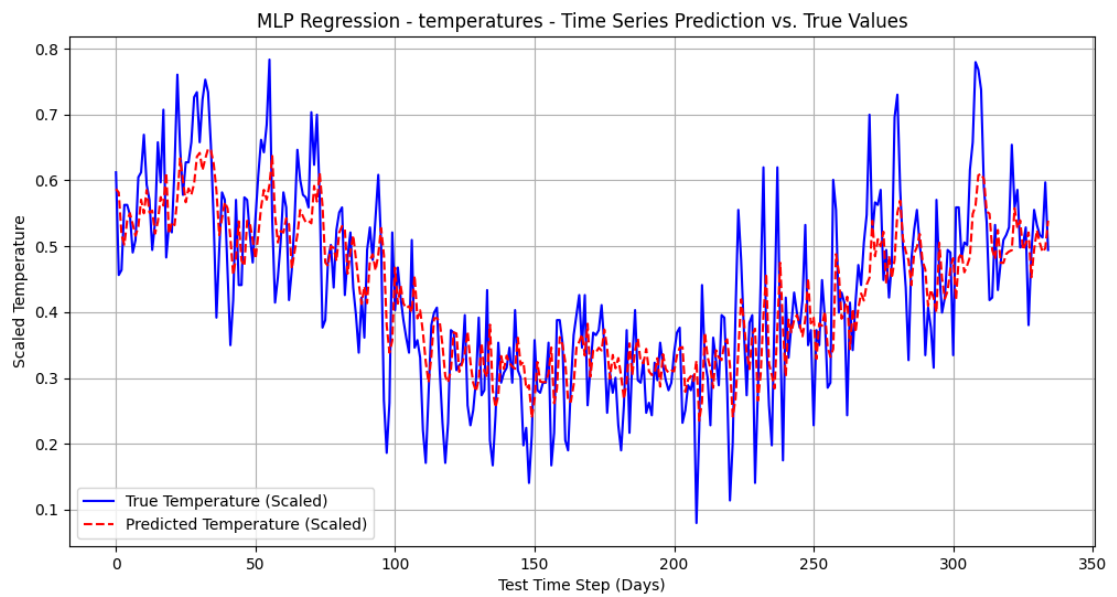


Figure 1: MLP

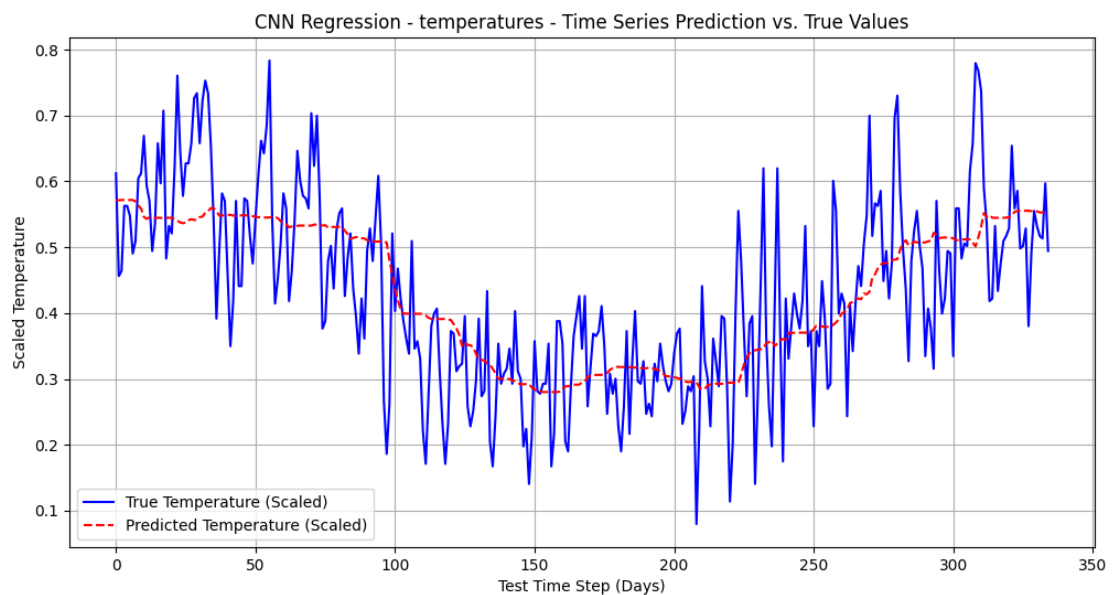


Figure 2: CNN

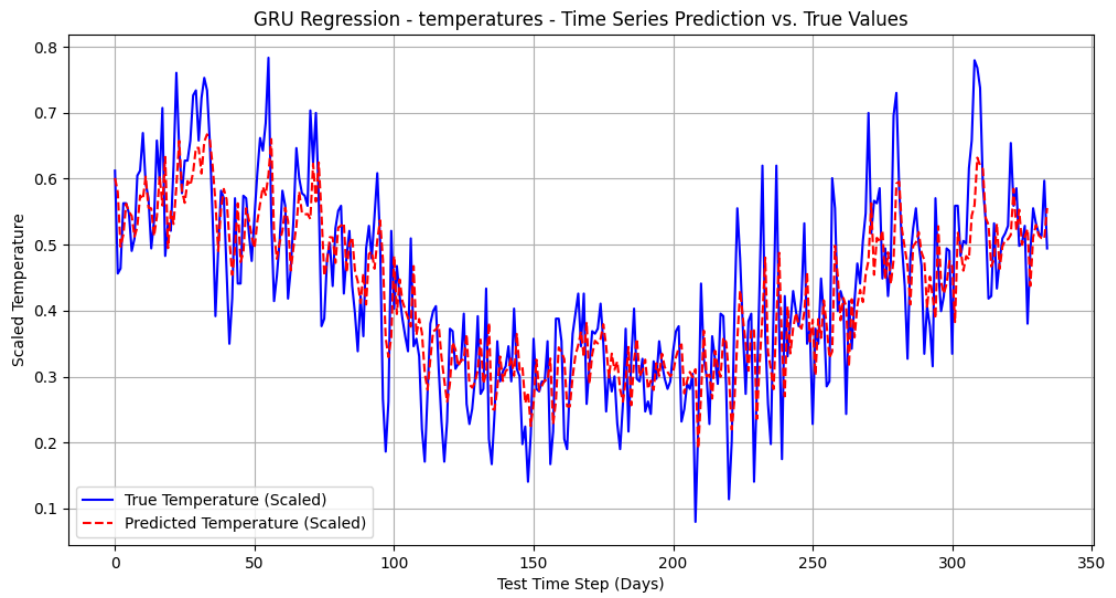


Figure 3: GRU

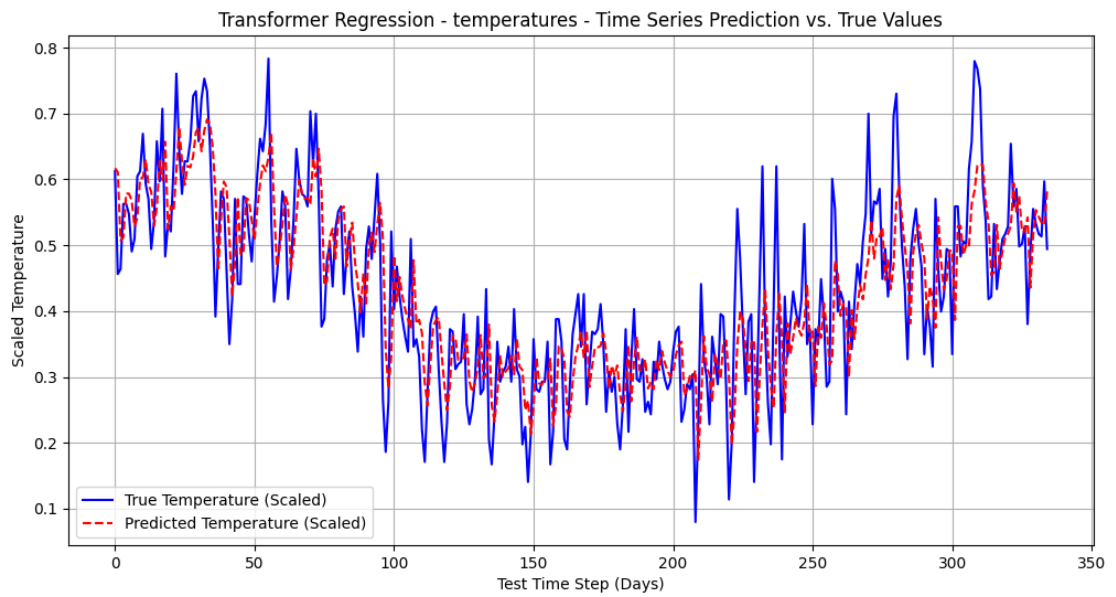


Figure 4: Transformer

B Plots of true value vs predicted for electric consumption dataset

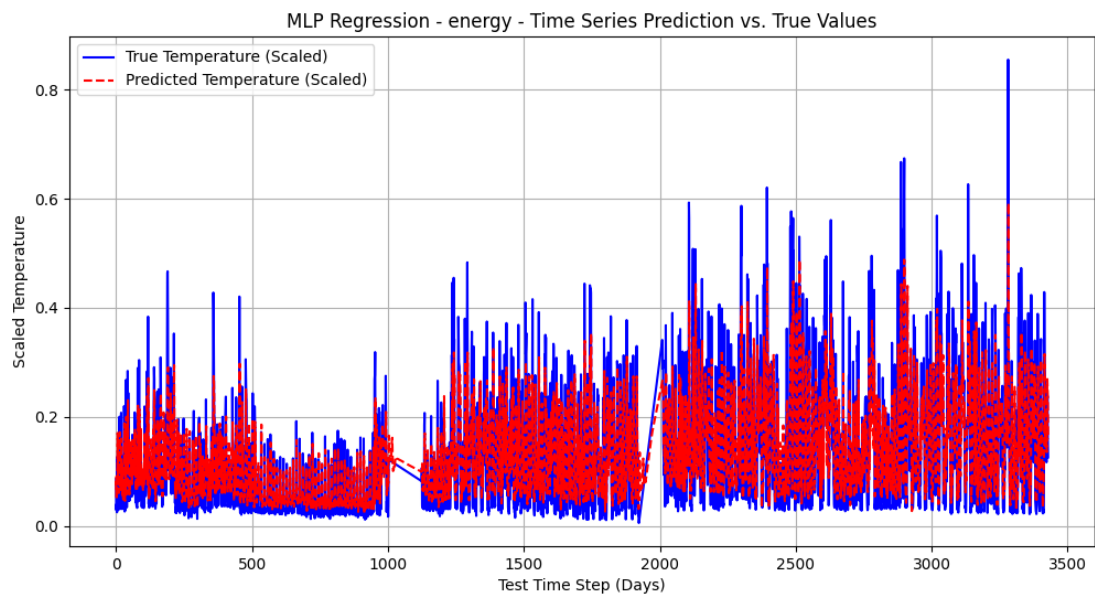


Figure 5: MLP

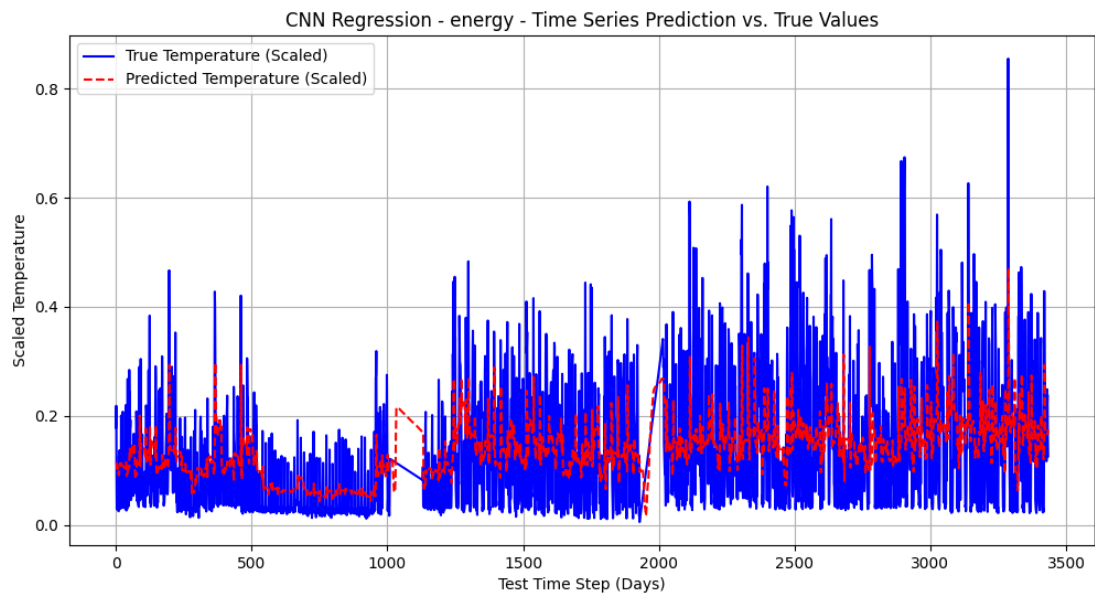


Figure 6: CNN

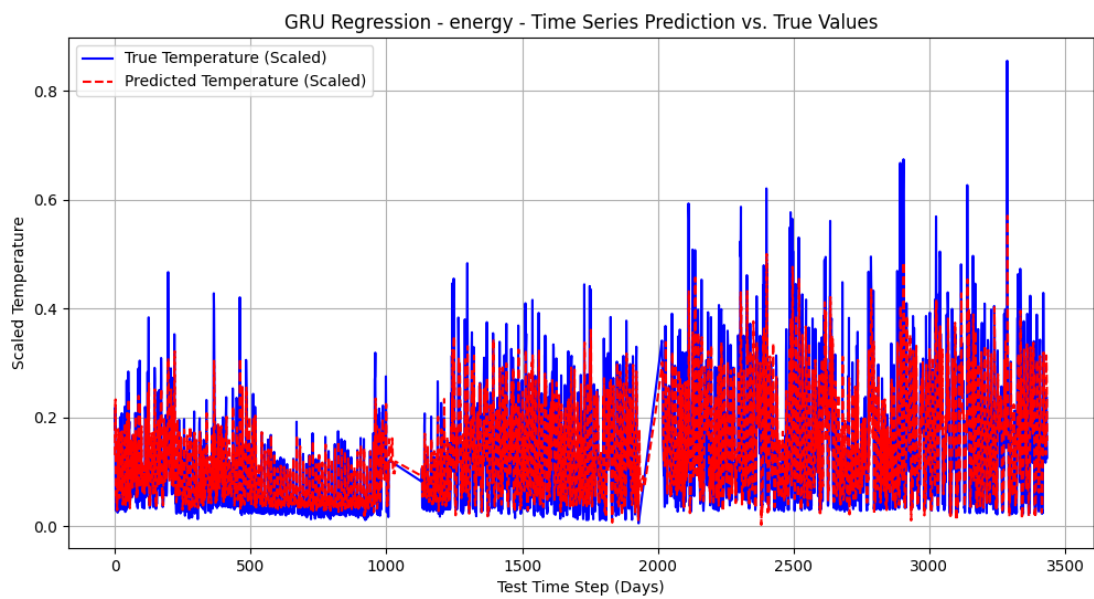


Figure 7: GRU

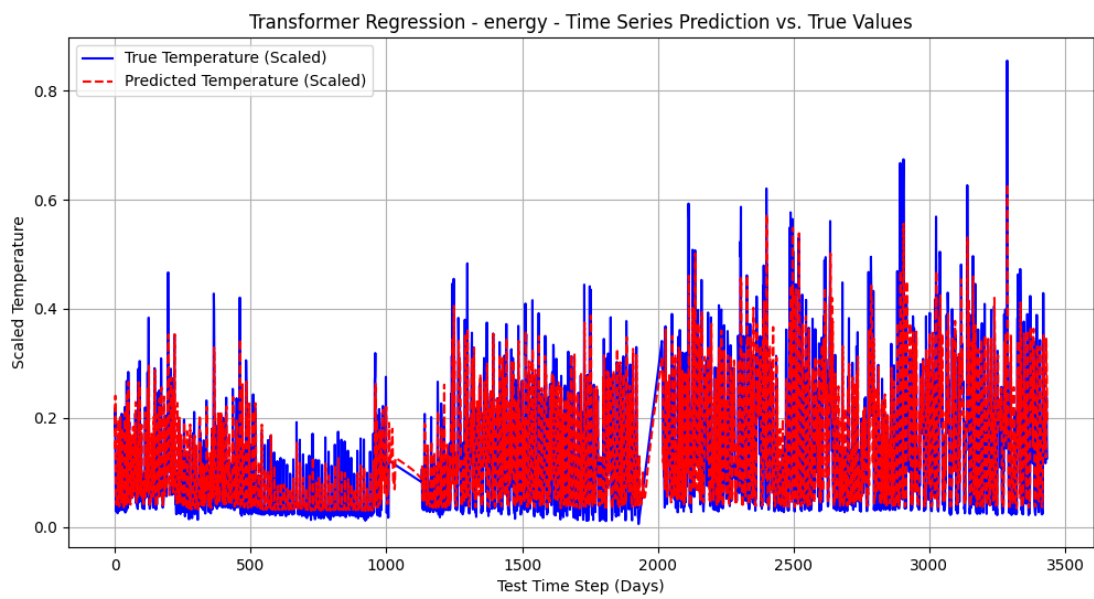


Figure 8: Transformer